Modul 9

Komunikasi Nirkabel dengan protokol ESP-NOW dan ESP-MESH

3.1 Tujuan

Setelah mengikuti praktikum Modul 9, mahasiswa diharapkan dapat:

1. Mengetahui dan Memahami Protokol ESP-NOW dan ESP-MESH.
2. Dapat Mengembangkan ESP-NOW dan ESP-MESH.
3. Praktikan dapat mengimplementasikan Protokol ESP-NOW dan ESP-MESH.

3.2 Alat dan Bahan

1. ESP32.
2. Laptop.
3. Software Arduino.
4. Kabel USB.

3.3.1 ESP NOW

ESP-NOW adalah protokol komunikasi nirkabel yang dikembangkan oleh Espressif Systems untuk penggunaan pada mikrokontroler ESP32. Protokol ini memungkinkan pengiriman data secara efisien dengan jarak yang cukup jauh menggunakan gelombang elektromagnetik pada frekuensi 2,4 GHz.

ESP-NOW memungkinkan perangkat berbasis ESP32 untuk saling bertukar data secara langsung melalui komunikasi nirkabel dalam mode peer-to-peer (P2P), Komunikasi dapat dilakukan tanpa membutuhkan router atau akses ke internet. Ini memungkinkan perangkat untuk saling bertukar informasi tanpa mengandalkan infrastruktur jaringan.

Keuntungan ESP-NOW:

1. **Low Power Consumption**: ESP-NOW sangat hemat daya, karena tidak memerlukan koneksi aktif ke Wi-Fi atau jaringan internet. Hal ini ideal untuk aplikasi IoT yang menggunakan perangkat berbasis baterai.
2. **Komunikasi cepat dan low latency**: Memungkinkan komunikasi antar perangkat dengan latensi rendah, cocok untuk aplikasi real-time seperti pengendalian perangkat dan monitoring sensor.
3. **No Need for Wi-Fi Router**: Karena ESP-NOW tidak memerlukan Wi-Fi atau jaringan lokal untuk komunikasi, ia bisa berfungsi bahkan dalam situasi tanpa akses internet atau router, seperti di area remote.
4. **Multicast Support**: ESP-NOW memungkinkan pengiriman data ke beberapa perangkat sekaligus, memudahkan distribusi data dalam aplikasi yang melibatkan banyak perangkat.
5. **Keamanan**: ESP-NOW mendukung enkripsi untuk memastikan data yang dikirimkan tetap aman, yang penting dalam aplikasi IoT yang membutuhkan perlindungan data.

Cara kerja ESP-NOW:

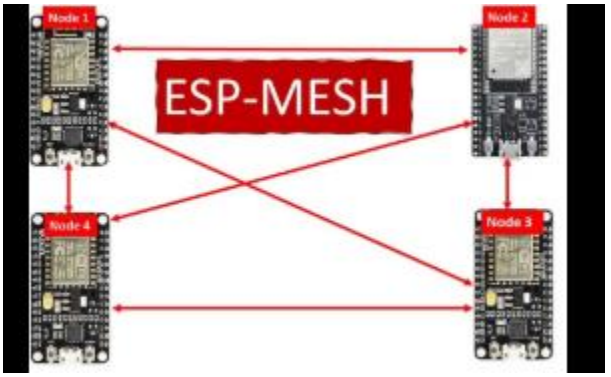
1. **Pengiriman Data**: Perangkat ESP32 dapat mengirim data ke perangkat lain yang sudah terhubung, menggunakan alamat MAC dari perangkat tujuan. ESP-NOW tidak bergantung pada IP atau pengalamatan berbasis jaringan lainnya.

2. **Mode Peer-to-Peer (P2P)**: ESP32 yang menggunakan ESP-NOW dapat saling terhubung langsung tanpa perantara. Data dapat dikirimkan dalam mode point-to-point (dari satu perangkat ke satu perangkat lain) atau point-to-multipoint (dari satu perangkat ke banyak perangkat).

3. **Addressing**: Setiap perangkat dalam jaringan ESP-NOW memiliki **alamat MAC** unik, dan komunikasi antar perangkat ditentukan berdasarkan alamat ini.

4. **Pengaturan Data**: Data yang dikirim dapat berupa paket-paket kecil dan sederhana (misalnya status sensor atau perintah kontrol perangkat). Biasanya, data dikemas dalam format binary untuk efisiensi.

3.3.2 ESP-MESH



ESP-MESH merupakan Protokol jaringan yang dibangun di atas Protokol Wi-Fi. ESP-MESH memungkinkan banyak perangkat (untuk selanjutnya disebut sebagai node) tersebar di area fisik yang luas (baik di dalam maupun di luar ruangan) untuk saling terhubung di bawah satu WLAN (Jaringan Area Lokal Nirkabel).

Dengan menghubungkan banyak perangkat dalam sebuah jaringan mesh, ESP-MESH memungkinkan jangkauan lebih luas, pengelolaan daya yang efisien, dan komunikasi yang lebih handal di seluruh jaringan. Protokol ini sangat cocok untuk aplikasi yang membutuhkan perangkat IoT saling berinteraksi dalam area yang luas dan dengan ketergantungan minimal pada infrastruktur pusat.

Konsep kerja:

1. **Routing Dinamis**: Jika salah satu perangkat (node) dalam jaringan tidak dapat terhubung langsung ke perangkat lainnya (misalnya karena jarak atau penghalang), perangkat-perangkat lain dalam jaringan bisa mengalihkan jalur data untuk memastikan komunikasi tetap berlangsung.

2. **Redundansi**: Jaringan mesh menawarkan **redundansi** atau jalur alternatif. Jika satu jalur komunikasi gagal, data dapat dialihkan melalui jalur lain yang tersedia, meningkatkan keandalan jaringan secara keseluruhan.

Cara kerja ESP-MESH:

1. **Node atau Perangkat**: Setiap perangkat yang terhubung ke jaringan ESP-MESH berfungsi sebagai node. Node ini bisa berupa perangkat pengumpul data, sensor, aktuator, atau bahkan perangkat yang berfungsi sebagai relay untuk perangkat lainnya.

2. **Node Akses Pusat**: Salah satu perangkat dalam jaringan bertindak sebagai **root node** atau node pusat, yang menghubungkan jaringan mesh ke jaringan eksternal, seperti internet. Namun, jaringan ESP-MESH tetap dapat berfungsi meskipun root node mengalami gangguan, karena jaringan tidak bergantung sepenuhnya pada satu perangkat.

3. **Jalur Pengiriman Data**: Data yang dikirimkan antara perangkat tidak selalu langsung menuju tujuan akhir. Data bisa "melompat" melalui beberapa perangkat (hops) hingga mencapai tujuan yang tepat.

4. **Pengelolaan Topologi**: Topologi jaringan dalam ESP-MESH bersifat dinamis, artinya perangkat baru bisa bergabung atau perangkat lama bisa keluar tanpa mengganggu keseluruhan sistem. Jaringan akan menyesuaikan dengan sendirinya.

3.3 Modul Praktikum

Langkah Peer to Peer ESP-NOW

- **Arduino Ide**

1. Siapkan alat dan bahan
2. Kita akan membuat codingan untuk mencari Mac Address,simpan codingan Mac Address karena akan kita gunakan kembali pada ESP-MESH, dibawah merupakan codingan Mac Address :

```
#include <WiFi.h>
#include <esp_wifi.h>

void readMacAddress(){
  uint8_t baseMac[6];
  esp_err_t ret = esp_wifi_get_mac(WIFI_IF_STA, baseMac);
  if (ret == ESP_OK) {
    Serial.printf("%02x:%02x:%02x:%02x:%02x:%02x\n",
                  baseMac[0], baseMac[1], baseMac[2],
                  baseMac[3], baseMac[4], baseMac[5]);
  } else {
    Serial.println("Failed to read MAC address");
  }
}

void setup(){
  Serial.begin(115200);

  WiFi.mode(WIFI_STA);
  WiFi.STA.begin();

  Serial.print("[DEFAULT] ESP32 Board MAC Address: ");
  readMacAddress();
}

void loop(){

}
```

3. Maka akan keluar hasil di Serial Monitor, jangan lupa tekan tombol N/RST atau Reset pada board ESP32 seperti di bawah ini:

4. Selanjutnya , membuat codingan sebagai sender .

```
#include <esp_now.h>
#include <WiFi.h>

// REPLACE WITH YOUR RECEIVER MAC Address
uint8_t broadcastAddress[] = {0xe4, 0x65, 0xb8, 0x83, 0xde, 0xac};


// Structure example to send data
// Must match the receiver structure
typedef struct struct_message {
  char a[32];
  int b;
  float c;
  bool d;
} struct_message;

// Create a struct_message called myData
struct_message myData;

esp_now_peer_info_t peerInfo;

// callback when data is sent
void OnDataSent(const uint8_t *mac_addr, esp_now_send_status_t status) {
  Serial.print("\r\nLast Packet Send Status:\t");
  Serial.println(status == ESP_NOW_SEND_SUCCESS ? "Delivery Success" : "Delivery Fail");
}

void setup() {
  // Init Serial Monitor
  Serial.begin(115200);
```

```
  // Set device as a Wi-Fi Station
  WiFi.mode(WIFI_STA);

  // Init ESP-NOW
  if (esp_now_init() != ESP_OK) {
    Serial.println("Error initializing ESP-NOW");
    return;
  }

  // Once ESPNow is successfully Init, we will register for Send CB to
  // get the status of Trasnmitted packet
  esp_now_register_send_cb(OnDataSent);

  // Register peer
  memcpy(peerInfo.peer_addr, broadcastAddress, 6);
  peerInfo.channel = 0;
  peerInfo.encrypt = false;

  // Add peer
  if (esp_now_add_peer(&peerInfo) != ESP_OK){
    Serial.println("Failed to add peer");
    return;
  }
}

void loop() {
  // Set values to send
  strcpy(myData.a, "THIS IS A CHAR");
  myData.b = random(1,20);
```

```
  memcpy(peerInfo.peer_addr, broadcastAddress, 6);
  peerInfo.channel = 0;
  peerInfo.encrypt = false;

  // Add peer
  if (esp_now_add_peer(&peerInfo) != ESP_OK){
    Serial.println("Failed to add peer");
    return;
  }
}

void loop() {
  // Set values to send
  strcpy(myData.a, "THIS IS A CHAR");
  myData.b = random(1,20);
  myData.c = 1.2;
  myData.d = false;

  // Send message via ESP-NOW
  esp_err_t result = esp_now_send(broadcastAddress, (uint8_t *) &myData, sizeof(myData));

  if (result == ESP_OK) {
    Serial.println("Sent with success");
  }
  else {
    Serial.println("Error sending the data");
  }
  delay(2000);
}
```

5. Maka akan keluar di serial monitor, jangan lupa tekan tombol N/RST atau Reset pada board ESP32 seperti dibawah ini :

```
Output   Serial Monitor ×
Message (Enter to send message to 'ESP32 Wrover Module' on 'COM13')

11:01:57.633 ->
11:01:57.633 -> Last Packet Send Status:        Delivery Success
11:01:59.640 -> Sent with success
11:01:59.640 ->
11:01:59.640 -> Last Packet Send Status:        Delivery Success
11:02:01.615 -> Sent with success
11:02:01.615 ->
11:02:01.615 -> Last Packet Send Status:        Delivery Success
11:02:03.615 -> Sent with success
11:02:03.615 ->
11:02:03.615 -> Last Packet Send Status:        Delivery Success
11:02:05.636 -> Sent with success
11:02:05.636 ->
11:02:05.636 -> Last Packet Send Status:        Delivery Success
11:02:07.617 -> Sent with success
11:02:07.617 ->
11:02:07.617 -> Last Packet Send Status:        Delivery Success
11:02:09.617 -> Sent with success
11:02:09.617 ->
11:02:09.617 -> Last Packet Send Status:        Delivery Success
11:02:11.618 -> Sent with success
11:02:11.618 ->
11:02:11.618 -> Last Packet Send Status:        Delivery Success
11:02:13.653 -> Sent with success
11:02:13.653 ->
11:02:13.653 -> Last Packet Send Status:        Delivery Success
11:02:15.616 -> Sent with success
11:02:15.616 ->
11:02:15.616 -> Last Packet Send Status:        Delivery Success
```

6. Selanjutnya codingan sebagai Receiver.

```
#include <esp_now.h>
#include <WiFi.h>

// Structure example to receive data
// Must match the sender structure
typedef struct struct_message {
    char a[32];
    int b;
    float c;
    bool d;
} struct_message;

// Create a struct_message called myData
struct_message myData;

// callback function that will be executed when data is received
void OnDataRecv(const uint8_t * mac, const uint8_t *incomingData, int len) {
  memcpy(&myData, incomingData, sizeof(myData));
  Serial.print("Bytes received: ");
  Serial.println(len);
  Serial.print("Char: ");
  Serial.println(myData.a);
  Serial.print("Int: ");
  Serial.println(myData.b);
  Serial.print("Float: ");
  Serial.println(myData.c);
  Serial.print("Bool: ");
  Serial.println(myData.d);
  Serial.println();
}

void setup() {
  // Initialize Serial Monitor
  Serial.begin(115200);

  // Set device as a Wi-Fi Station
  WiFi.mode(WIFI_STA);

  // Init ESP-NOW
  if (esp_now_init() != ESP_OK) {
    Serial.println("Error initializing ESP-NOW");
    return;
  }

  // Once ESPNow is successfully Init, we will register for recv CB to
  // get recv packer info
  esp_now_register_recv_cb(esp_now_recv_cb_t(OnDataRecv));
```

```
}

void loop() {

}
```

7. Maka akan keluar hasil di serial monitor.

```
COM18                                                    Send
Bytes received: 44
Char: THIS IS A CHAR
Int: 10
Float: 1.20
Bool: 0

Bytes received: 44
Char: THIS IS A CHAR
Int: 15
Float: 1.20
Bool: 0

Bytes received: 44
Char: THIS IS A CHAR
Int: 17
Float: 1.20
Bool: 0

Bytes received: 44
Char: THIS IS A CHAR
Int: 14
Float: 1.20
Bool: 0
```

Langkah-Langkah ESP-MESH

8. Untuk Mac Address, Copy Mac Address yang telah kalian simpan tadi.
9. Buat Codingan sender:

```
//Sender 1
#include <esp_now.h>
#include <WiFi.h>

// REPLACE WITH THE RECEIVER'S MAC Address
uint8_t broadcastAddress[] = {0xB0, 0xB2, 0x1C, 0x51, 0xDE, 0x34};

// Structure example to send data
// Must match the receiver structure
typedef struct struct_message {
    int id; // must be unique for each sender board
    int x;
    int y;
} struct_message;

// Create a struct_message called myData
struct_message myData;

// Create peer interface
esp_now_peer_info_t peerInfo;

// callback when data is sent
void OnDataSent(const uint8_t *mac_addr, esp_now_send_status_t status) {
  Serial.print("\r\nLast Packet Send Status:\t");
  Serial.println(status == ESP_NOW_SEND_SUCCESS ? "Delivery Success" : "Delivery Fail");
}

void setup() {
  // Init Serial Monitor
  Serial.begin(115200);

  // Set device as a Wi-Fi Station
  WiFi.mode(WIFI_STA);

  // Init ESP-NOW
  if (esp_now_init() != ESP_OK) {
    Serial.println("Error initializing ESP-NOW");
    return;
  }

  // Once ESPNow is successfully Init, we will register for Send CB to
  // get the status of Trasnmitted packet
  esp_now_register_send_cb(OnDataSent);

  // Register peer
  memcpy(peerInfo.peer_addr, broadcastAddress, 6);
  peerInfo.channel = 0;
```

```
  peerInfo.encrypt = false;

  // Add peer
  if (esp_now_add_peer(&peerInfo) != ESP_OK){
    Serial.println("Failed to add peer");
    return;
  }
}

void loop() {
  // Set values to send
  myData.id = 1;
  myData.x = random(0,50);
  myData.y = random(0,50);

  // Send message via ESP-NOW
  esp_err_t result = esp_now_send(broadcastAddress, (uint8_t *) &myData, sizeof(myData));

  if (result == ESP_OK) {
    Serial.println("Sent with success");
  }
  else {
    Serial.println("Error sending the data");
  }
  delay(10000);
}
```

10. Maka akan keluar serial monitor, jangan lupa tekan tombol N/RST atau Reset pada board ESP32

```
COM18                                                    Send
Sent with success

Last Packet Send Status:        Delivery Fail
Sent with success

Last Packet Send Status:        Delivery Fail
Sent with success

Last Packet Send Status:        Delivery Success
Sent with success

Last Packet Send Status:        Delivery Success
Sent with success

Last Packet Send Status:        Delivery Success
Sent with success

Last Packet Send Status:        Delivery Success
Sent with success

Last Packet Send Status:        Delivery Success
Sent with success

Last Packet Send Status:        Delivery Success
```

11. Buat codingan Receiver

ESP32_NOW_Receiver.ino

```
//RECEIVER//
#include <esp_now.h>
#include <WiFi.h>

// Structure example to receive data
// Must match the sender structure
typedef struct struct_message {
  int id;
  int x;
  int y;
}struct_message;

// Create a struct_message called myData
struct_message myData;

// Create a structure to hold the readings from each board
struct_message board1;
struct_message board2;
struct_message board3;

// Create an array with all the structures
struct_message boardsStruct[3] = {board1, board2, board3};

// callback function that will be executed when data is received
void OnDataRecv(const uint8_t * mac_addr, const uint8_t *incomingData, int len) {
  char macStr[18];
  Serial.print("Packet received from: ");
  snprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",
           mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
  Serial.println(macStr);
  memcpy(&myData, incomingData, sizeof(myData));
  Serial.printf("Board ID %u: %u bytes\n", myData.id, len);
  // Update the structures with the new incoming data
  boardsStruct[myData.id-1].x = myData.x;
  boardsStruct[myData.id-1].y = myData.y;
  Serial.printf("x value: %d \n", boardsStruct[myData.id-1].x);
  Serial.printf("y value: %d \n", boardsStruct[myData.id-1].y);
  Serial.println();
}

void setup() {
  //Initialize Serial Monitor
  Serial.begin(115200);

  //Set device as a Wi-Fi Station
  WiFi.mode(WIFI_STA);

  //Init ESP-NOW
  if (esp_now_init() != ESP_OK) {
    Serial.println("Error initializing ESP-NOW");
    return;
  }

  // Once ESPNow is successfully Init, we will register for recv CB to
  // get recv packer info
  esp_now_register_recv_cb(esp_now_recv_cb_t(OnDataRecv));
}
```

```
void loop() {
  // Acess the variables for each board
  /*int board1X = boardsStruct[0].x;
  int board1Y = boardsStruct[0].y;
  int board2X = boardsStruct[1].x;
  int board2Y = boardsStruct[1].y;
  int board3X = boardsStruct[2].x;
  int board3Y = boardsStruct[2].y;*/

  delay(10000);
}
```

12. Maka akan muncul serial monitor seperti di bawah ini:

```
13:10:05.524 -> Packet received from: de:f3:fa:3f:d8:f3
13:10:05.524 -> Board ID 2: 12 bytes
13:10:05.524 -> x value: 11
13:10:05.524 -> y value: 1
13:10:05.524 ->
13:10:15.519 -> Packet received from: de:f3:fa:3f:d8:f3
13:10:15.519 -> Board ID 2: 12 bytes
13:10:15.519 -> x value: 9
13:10:15.519 -> y value: 11
13:10:15.519 ->
13:10:25.507 -> Packet received from: de:f3:fa:3f:d8:f3
13:10:25.507 -> Board ID 2: 12 bytes
13:10:25.507 -> x value: 48
13:10:25.507 -> y value: 6
13:10:25.507 ->
13:10:35.479 -> Packet received from: de:f3:fa:3f:d8:f3
13:10:35.519 -> Board ID 2: 12 bytes
13:10:35.519 -> x value: 12
13:10:35.519 -> y value: 49
```

SELAMAT MENGERJAKAN